

September 2026 DSA Training Plan

Primary Goal: Go from "DSA goes over my head" to being able to independently recognize and solve common Easy/Medium Graph problems and understand foundational Dynamic Programming.

Dates: September 2--30, 2026

Daily DSA Budget: ~2 hours

Primary Teacher: Striver

Primary Focus: Graphs → Dynamic Programming

Secondary Goal: Build the recursion/traversal foundations needed to make Graphs and DP understandable.

Resource Stack

Primary Graph Resource

Striver Graph Series (56 videos)

<https://www.youtube.com/playlist?list=PLgUwDviBlf0oE3gA41TKO2H5bHpPd7fzn>

This is the backbone of the Graph block. Follow the playlist in order, but **do not optimize for finishing videos**.

Primary Practice / Syllabus

Striver A2Z DSA Sheet

<https://takeuforward.org/dsa/strivers-a2z-sheet-learn-dsa-a-to-z>

Use the corresponding problems from the sheet while progressing through the concepts.

Backup Explanation

NeetCode DSA for Beginners

<https://neetcode.io/courses/dsa-for-beginners/0>

Use NeetCode only when a Striver explanation genuinely does not click. Do not resource-hop by default.

Practice Platform

LeetCode

<https://leetcode.com/>

Use LeetCode for independent attempts, re-solves, and assessment problems.

Core Learning Rule

Watching a video is **not** completing a topic.

For every concept/problem:

1. **Learn the concept.**
2. **Pause before the solution if possible.**
3. **Attempt the problem independently.**
4. If stuck, take a small hint.
5. If still stuck, study the solution.
6. Close the video/solution.
7. Re-code the complete solution from an empty editor.
8. Explain why the algorithm works and its time/space complexity.
9. Re-solve important problems later without looking at old code.

A problem counts as **learned** only when you can reconstruct the reasoning later.

Daily Session Structure

Standard 2-Hour Session

Block 1 --- Learn: 25--35 min

Watch the relevant Striver lesson.

Do not passively binge videos.

Write down: - What problem is this algorithm solving? - What is the intuition? - What data structure is involved? - What is the time complexity?

Block 2 --- Independent Attempt: 30--40 min

Attempt the associated problem without AI or the solution.

Before coding, identify: - Input - Desired output - Brute-force idea - Pattern/algorithm - Time and space complexity

Block 3 --- Learn + Reconstruct: 25--35 min

If stuck, finish Striver's explanation.

Then **close the solution and code it yourself from scratch.**

Block 4 --- Revision: 15--25 min

Re-solve or verbally derive one older problem.

Approximate revision cycle:

Day 0 → Day +2 → Day +7 → Day +14

WEEK 1 --- Sep 2--7

FOUNDATION + GRAPH ENTRY

Purpose: Build enough recursion, traversal, queue, and graph-representation understanding that the Graph playlist does not become memorization.

The goal is not volume. The goal is getting comfortable with recursive thinking and traversal.

Sep 2 --- Complexity, Hashing & DSA Reset

Learn

- Big-O intuition
- $O(1)$, $O(n)$, $O(n \log n)$, $O(n^2)$
- HashMap / HashSet
- Why additional memory can reduce runtime

Problems

- Contains Duplicate
- Two Sum

Goal

Be able to explain why hashing turns Two Sum from an $O(n^2)$ search into an $O(n)$ approach.

Sep 3 --- Recursion Fundamentals

Learn

- Base case
- Recursive call
- Call stack
- Recursion tree
- Returning values through recursive calls

Implement Yourself

- Factorial
- Fibonacci
- Reverse an array/string recursively

Goal

Draw the call stack/recursion tree on paper instead of memorizing templates.

Sep 4 --- Recursive Traversal

Learn

- DFS intuition
- Recursive exploration
- visited state
- Why recursion naturally fits traversal

Practice

Use a simple tree or graph and manually perform DFS.

Optional easy tree problems: - Maximum Depth of Binary Tree - Same Tree

Goal

Understand the generic idea:

```
visit node → visit unexplored neighbours/children →  
return
```

Sep 5 --- BFS + Queue

Learn

- Queue
- FIFO
- Level-by-level traversal
- BFS vs DFS intuition

Practice

- Binary Tree Level Order Traversal
- Manually BFS a small graph

Goal

Understand why BFS explores outward layer by layer.

Sep 6 --- Start Striver Graph Series

Playlist Focus

Begin the Striver Graph Series from the start.

Cover: - Graph introduction - Graph representation - Adjacency matrix - Adjacency list

Practice

Create a graph manually and implement its adjacency list.

Example:

```
0 → 1, 2
1 → 0, 3
2 → 0, 3
3 → 1, 2
```

Goal

Know what nodes and edges actually become in code.

Sep 7 --- DFS/BFS on Graphs

Continue the playlist into: - Graph BFS - Graph DFS

Practice

Implement generic BFS and DFS without copying.

Then attempt one simple traversal problem.

Weekly Checkpoint

By the end of Week 1, you should be able to explain:

- What an adjacency list is
- What `visited[]` does
- DFS
- BFS
- Stack/recursion vs queue
- Why disconnected graphs may require starting traversal multiple times

If these are unclear, repeat them before moving ahead.

WEEK 2 --- Sep 8--14

GRAPH FUNDAMENTALS

Purpose: Make DFS/BFS-based graph problems familiar.

Continue following the Striver Graph playlist in order. Do not worry about the number of videos completed.

Sep 8 --- Connected Components

Focus

- Multiple components
- Traversing disconnected graphs
- Component counting

Practice

- Number of Provinces / equivalent Striver problem

Recognition Pattern

When asked "how many groups/networks/components?" think traversal + visited.

Sep 9 --- Grid Graphs

Focus

Understand that a matrix can be treated as a graph.

Practice

- Number of Islands

Goal

Recognize: - Cell = node - Adjacent cells = edges

Sep 10 --- More Grid Traversal

Practice

- Flood Fill
- Rotting Oranges

Focus

Understand when BFS is useful for simultaneous/layered spreading.

Sep 11 --- DFS/BFS Consolidation

Practice

- Max Area of Island or equivalent
- Re-solve Number of Islands without seeing previous code

Goal

You should start recognizing traversal problems before seeing the solution.

Sep 12 --- Cycle Detection: Undirected Graph

Learn

- What a graph cycle is
- Parent tracking
- BFS/DFS cycle detection

Practice

One Striver cycle-detection problem.

Goal

Understand *why* seeing an already-visited neighbour is not automatically a cycle in an undirected graph.

Sep 13 --- Cycle Detection: Directed Graph

Learn

- Directed cycles
- DFS path state / recursion stack concept

Practice

One directed-cycle problem from the playlist.

Sep 14 --- GRAPH CHECKPOINT #1

No new algorithm initially.

Test

Attempt: 1. One unseen DFS/BFS problem 2. One unseen grid problem

Maximum ~35--40 minutes each.

Then revisit your weakest Week 2 problem.

Success Criteria

Before coding a basic graph problem, you should naturally ask:

- What are the nodes?
 - What are the edges?
 - Is this directed?
 - Do I need visited state?
 - BFS or DFS?
 - Is this one component or multiple?
-

WEEK 3 --- Sep 15--21

ADVANCED GRAPH PATTERNS

Purpose: Progress from traversal into the major interview graph algorithms.

Continue through the Striver Graph Series rather than manually jumping between unrelated topics.

Sep 15 --- Topological Sort

Learn

- DAG
- Dependency ordering
- Indegree
- Kahn's algorithm
- DFS topological ordering intuition

Practice

- Course Schedule or equivalent

Recognition Pattern

Dependencies / prerequisites / ordering constraints → consider topological sort.

Sep 16 --- Topological Sort Practice

Practice

- Course Schedule II or equivalent
- One additional Striver topo-sort problem if time allows

Goal

Understand why topological sorting is only valid for DAGs.

Sep 17 --- Shortest Path Fundamentals

Learn

Distinguish:

Unweighted graph → BFS can give shortest path

Positive weighted graph → Dijkstra

Practice

Implement/trace an easy shortest-path example manually.

Do not rush into memorizing Dijkstra code.

Sep 18 --- Dijkstra

Learn

- Priority queue / min-heap
- Distance array
- Relaxation

- Why greedy extraction works with non-negative weights

Practice

- Network Delay Time or equivalent Striver problem

Goal

Be able to explain what `dist[node]` means.

Sep 19 --- Disjoint Set Union (DSU)

Learn

- `parent[]`
- `find()`
- `union()`
- Path compression
- Union by rank/size

Practice

- Redundant Connection or equivalent

Goal

Implement a basic DSU yourself.

One good problem is enough today.

Sep 20 --- Minimum Spanning Tree

Learn

- What an MST is
- Prim's intuition
- Kruskal's intuition
- How Kruskal connects to DSU

Practice

One basic MST problem from Striver.

Goal

Conceptual understanding > memorizing every implementation detail.

Sep 21 --- GRAPH EXAM

No tutorials before attempting.

Test

Pick three unseen or previously untouched problems:

1. DFS/BFS or grid traversal
2. Topological sort / DSU
3. Shortest path

Give yourself approximately 35 minutes per problem.

Record each as:

- **A --- Solved independently**
- **B --- Correct approach, implementation issue**
- **C --- Needed a hint**
- **D --- Could not identify approach**

Graph Milestone

Target status:

Area Target

DFS/BFS Comfortable Grid graphs Comfortable Connected components
Comfortable Cycle detection Familiar Topological sort Understand +
basic application Dijkstra Understand + basic application DSU
Understand MST Conceptually familiar

You do **not** need mastery of every Graph playlist video before moving to DP.

WEEK 4 --- Sep 22--28

DYNAMIC PROGRAMMING

Purpose: Build DP from recursion rather than memorizing tables.

Use Striver's DP material with the same learning loop used for Graphs:
concept → attempt → solution if required → reconstruct → revisit.

The objective is foundational DP understanding, not completing the entire DP series.

Sep 22 --- Why DP Exists

Start With Fibonacci

Implement: 1. Plain recursion 2. Draw recursion tree 3. Identify repeated work 4. Memoization 5. Tabulation

Problem

- Climbing Stairs

Core Idea

DP is primarily about solving overlapping subproblems once and reusing their answers.

Sep 23 --- 1D DP

Learn

- State
- Transition / recurrence
- Base case
- Memoization vs tabulation

Problems

- Climbing Stairs --- re-solve
- Frog Jump / Min Cost Climbing Stairs

Rule

Before coding, write in English:

`dp[i]` represents _____.

Sep 24 --- Decision-Based 1D DP

Problem

- House Robber

If comfortable: - House Robber II

Goal

Derive the decision:

`take current` vs `skip current`

Then derive what previous states each choice depends on.

Sep 25 --- Grid DP

Learn

- 2D states
- Movement choices
- Recurrence across rows/columns

Problems

- Unique Paths
- Minimum Path Sum if time allows

Goal

Understand what each cell of the DP table represents.

Sep 26 --- Subsequence / Knapsack Introduction

Learn

The fundamental choice:

take vs not take

Practice

- Basic 0/1 Knapsack from Striver
- Partition Equal Subset Sum if ready

This is expected to feel harder.

Do not force multiple problems if the state/recurrence does not make sense yet.

Sep 27 --- String DP

Focus

Longest Common Subsequence.

Practice

- LCS

Draw the DP table manually.

Goal

Understand why matching and non-matching characters produce different transitions.

One deeply understood LCS problem is enough.

Sep 28 --- DP Consolidation

Choose Based on Weakness

Revisit two of: - House Robber - Unique Paths - Knapsack - Partition Equal Subset Sum - LCS

Do not add another difficult DP pattern simply to stay on schedule.

DP Checkpoint

Before coding, force yourself to answer:

1. What is changing?
2. What information defines my state?
3. What decision am I making?

4. What smaller states does this depend on?
5. What is the base case?

Only then code.

Sep 29 --- DP EXAM

No tutorials initially.

Test

Attempt: 1. One unseen/basic 1D DP problem 2. One unseen grid/2D DP problem

Then re-solve one previously difficult DP problem from an empty editor.

Record

A / B / C / D using the same grading system as the Graph exam.

Sep 30 --- FINAL DSA ASSESSMENT

Treat this as the equivalent of a race/test session.

No ChatGPT.

No Striver.

No old code.

No solution videos during the initial attempt.

Problems

Attempt four interview-style problems:

1. One Array/Hashing problem
2. One DFS/BFS Graph problem
3. One Graph algorithm problem
4. One foundational DP problem

Give yourself approximately 30--35 minutes per problem.

Grade

Grade Meaning

A Solved independently B Correct algorithm, implementation issue C Needed a hint D Could not identify the approach

The result determines what you revise after your vacation.

September At a Glance

Block Dates Focus Main Outcome

Foundation Sep 2--7 Recursion, Graph DFS/BFS, prerequisites representation make sense

Graph Build 1 Sep 8--14 Traversal, grids, Basic Graph components, problems become cycles recognizable

Graph Build 2 Sep 15--21 Topo, shortest Major Graph path, DSU, MST patterns become familiar

DP Build Sep 22--28 Recursion → State and memoization → recurrence begin foundational DP to make sense

Assessment Sep 29--30 DP + mixed Measure testing independent problem-solving

Problem Volume Target

Do **not** target 100 problems.

A reasonable September target is approximately:

- Foundation: 6--8
- Graphs: 18--25
- DP: 10--15
- Assessments/re-solves: 8--12

Total attempts: roughly 40--55, including re-solves.

The goal is not the number.

The important metric is how many problems you can later solve or derive **without the teacher**.

DSA Scoreboard

Track these daily:

Metric Track

DSA session completed Yes / No New problems attempted Number Solved independently Number Solved after hint Number Required full solution Number Revision problems Number Main concept learned Short note

The most useful long-term metric is:

Independent solves ÷ total attempts

The ratio should gradually increase.

Modification Rules

If a concept goes completely over your head

Do **not** continue watching five more videos.

1. Replay/rewind the important section.
 2. Draw the example manually.
 3. Use NeetCode as a second explanation.
 4. Attempt an easier example.
 5. Return to Striver.
-

If you are stuck on a problem

Easy

Give yourself ~20--25 minutes.

Medium

Give yourself ~30--40 minutes.

Then: 1. Take one hint. 2. Try again. 3. If still stuck, learn the solution. 4. Close it. 5. Reproduce it yourself.

Getting help is fine.

Copying a solution and marking the problem complete is not.

If you miss a day

Do not compensate with a four-hour grind the next day.

Continue the plan and compress/remove a lower-priority problem later.

Consistency matters more than perfect calendar adherence.

If Graphs take longer than expected

Take time away from advanced Graph breadth, not from DFS/BFS fundamentals.

Priority:

DFS/BFS → grids/components → topo → shortest path → DSU → MST

If DP takes longer than expected

Do not rush through 15 DP patterns.

Priority:

recursion → **memoization** → **1D DP** → **grid DP** → **take/not-take**

Understanding those is enough for this first block.

If mentally exhausted

Use a 45--60 minute maintenance session:

- Re-solve one known problem
- Review one concept
- Explain one algorithm aloud

This is the DSA equivalent of an easy/recovery training day.

AI Usage Rule

During DSA, AI is a **tutor**, not a solver.

Do not ask:

Solve this problem for me.

Instead use:

I am learning DSA. Do not give me the solution or code. Ask me questions and give me one progressively stronger hint at a time until I

derive the approach myself.

If you eventually need the solution, study it, close it, and reconstruct it independently.

What Success Looks Like on Sep 30

Success is **not**:

- Finishing all 56 Graph videos
- Completing the entire Striver DP series
- Solving 100 LeetCode questions
- Never needing hints

Success means:

Graphs

When you see a problem, you can ask:

What are the nodes and edges? Is this traversal, components, ordering, connectivity, or shortest path?

and choose a plausible algorithm.

DP

When you see a basic DP problem, you can ask:

What is my state? What choices do I have? What smaller states does this depend on?

and derive at least a recursive/memoized direction.

The One Thing

By Sep 21, DFS/BFS and common Graph problems should no longer feel alien.

By Sep 30, you should be able to define the state and recurrence for a basic DP problem without immediately opening a solution.

Everything else is secondary.

After the October vacation, transition from concentrated learning into mixed interview preparation such as NeetCode 150 / LeetCode-style pattern practice while continuing to revisit Graphs and DP.